PRODUCT MANAGEMENT SYSTEM

Backend Architecture & System Documentation

TABLE OF CONTENTS

1. System Overview
2. Technology Stack
3. Database Design & Models
4. Authentication & Security
5. API Endpoints Overview
6. Standard Request / Response Format
7. Environment & Deployment Configuration

1. SYSTEM OVERVIEW

The backend is a RESTful API service built for a Product Management ERP system. It handles user authentication, product catalog management, category organization, inventory tracking, image uploads via Cloudinary, and dashboard analytics aggregation. The service operates with a NeonDB (PostgreSQL) relational database using Prisma ORM.

2. TECHNOLOGY STACK

- Runtime: Node.js (ES Modules, "type": "module")
- Web Framework: Express 5
- Database: PostgreSQL (Neon Serverless)
- ORM: Prisma Client 5.22
- Authentication: JWT (jsonwebtoken) + bcryptjs password hashing
- Validation: Joi schema validation
- File Storage: Multer (memory storage) + Cloudinary SDK
- Security Middlewares: Helmet, Express Rate Limit, Express Sanitizer, CORS

3. DATABASE DESIGN & MODELS

The PostgreSQL database consists of 3 relational models managed by Prisma:

3.1 User (Table: "users")

- userId: UUID (Primary Key)
- userName: String
- email: String (Unique)
- password: String (Hashed with bcrypt)
- role: Role Enum (ADMIN, STAFF; Default: ADMIN)
- refreshToken: Text (Nullable)
- createdAt, updatedAt: Timestamps

3.2 Category (Table: "categories")

- categoryId: UUID (Primary Key)
- name: String (Unique)
- description: Text (Nullable)

- Relation: Has many Products (1-to-N, delete restricted if products exist)
- createdAt, updatedAt: Timestamps

3.3 Product (Table: "products")

- productId: UUID (Primary Key)
- sku: String (Unique, Indexed)
- name: String (Indexed)
- description: Text (Nullable)
- brand: String (Nullable)
- costPrice: Decimal(10,2)
- sellingPrice: Decimal(10,2)
- stockQuantity: Integer (Default: 0, Indexed)
- lowStockThreshold: Integer (Default: 10)
- unit: String (Default: "pcs")
- status: ProductStatus Enum (ACTIVE, INACTIVE, OUT_OF_STOCK, DISCONTINUED)
- imageUrl: Text (Nullable)
- imagePublicId: Text (Nullable, Cloudinary identifier)
- categoryId: UUID (Foreign Key -> categories.categoryId, Indexed)
- userId: UUID (Foreign Key -> users.userId, Cascade delete)
- createdAt, updatedAt: Timestamps

4. AUTHENTICATION & SECURITY

- Password Hashing: Salted bcrypt hashing before storing in database.
- Access Tokens: Short-lived JWT tokens passed via Authorization header ("Bearer <token>").
- Refresh Tokens: Long-lived tokens stored in the database and sent via custom "x-refresh-token" header to rotate access tokens without forcing re-login.
- Route Protection: "isAuthorized" middleware validates token signature, expiration, and user existence before executing protected controller methods.
- Rate Limiting: 100 requests per second per IP to prevent brute-force and DoS attacks.
- Input Sanitization & Validation: Express Sanitizer strips script injection; Joi validates data types and constraints on every incoming request.
- Image Handling: Multer handles multipart in-memory buffers; images are uploaded to Cloudinary, and older images are deleted upon product update/removal.

5. API ENDPOINTS OVERVIEW

All endpoints are RESTful and versioned under the /api/v1 prefix:

- Authentication (/api/v1/Auth): User registration, credential login, token rotation (/RefreshToken), and session logout.
- Category Management (/api/v1/Category): CRUD operations for product categories including paginated search/listing and dropdown options.
- Product Catalog (/api/v1/Product): Complete catalog management with server-side pagination, multi-field search and filters, Cloudinary image upload, inline status toggling, and deletion.
- Dashboard Analytics (/api/v1/Dashboard): Aggregated inventory statistics (/Stats) providing summary counts, top 4 categories distribution, and low-stock replenishment alerts.

6. STANDARD REQUEST / RESPONSE FORMAT

All API responses follow a unified JSON envelope:

Success Response:

```
{
  "status": 1,
  "message": "<Status description message>",
  "data": { ... }
}
```

Failure / Error Response:

```
{
  "status": 0,
  "message": "<Error description message>",
  "error": "<Technical error detail if available>"
}
```

7. ENVIRONMENT & DEPLOYMENT CONFIGURATION

Required environment variables (.env):

- PORT: Application port (Default: 5000)
- DATABASE_URL: Neon PostgreSQL connection pooler string
- DIRECT_URL: Direct Neon PostgreSQL connection string (for migrations)
- JWT_SECRET: Secret key for signing access tokens
- JWT_REFRESH_SECRET: Secret key for signing refresh tokens
- CLOUDINARY_CLOUD_NAME, CLOUDINARY_API_KEY, CLOUDINARY_API_SECRET: Cloudinary credentials
- ALLOWED_ORIGIN: Frontend URL for CORS configuration